Validation finale — MediConnect

Classe 5TWIN · Équipe : Chemlali Ismail (patients, back-office), Missaoui Mourad (médecins), Gobji Walid (infirmiers), Allaya Ahmed (pharmacies) · chaque critère de la grille et sa preuve

1 386	89 %	100/100	95 %	291
tests automatisés, tous verts	de couverture (lignes + branches)	Lighthouse accessibilité, 13 pages / 5 rôles	exactitude règles + IA (alertes de soins)	pages parcourues en production Docker, 0 erreur

Git et intégration conditions de la note

Exigence	Preuve
Git OUI	Une branche par module (`gestion-patient` , `gestion-medecins` , `gestion-infirmiers` , `Pharmacie`), intégration dans `preprod` par Pull Requests relues, conflits résolus et documentés dans chaque PR.
Intégration & déploiement OUI	Intégration continue GitHub Actions (MongoDB + Redis, tests, `check --deploy` , couverture) ; image `Dockerfile` unique ; `docker compose up` (site, worker Celery, Redis, MongoDB) ; en ligne : Railway (site, worker, Redis) + MongoDB Atlas, Vercel comme porte d'entrée HTTPS.

1 · Fonctionnalités & CRUD 2 points

Module	Entités (CRUD par les pages)	Au-delà du CRUD
Patients	Patient, DossierMedical, Allergie, Mesure, Triage	Triage IA niveau 1→5, file équitable, agenda, export de ses données
Médecins	Medecin, Consultation, Prescription	File triée, requalification, visio, compte rendu PDF, e-mail / SMS / WhatsApp
Infirmiers	Infirmier, InterventionDomicile	Zones, séries de soins, planning sans conflit, tournée, compte rendu intelligent, alertes
Pharmacies	Pharmacie, Ordonnance, Medicament	Stock, rupture, délivrance partielle, substitution IA, extraction IA des médicaments
Comptes	User (5 rôles), JournalConnexion	Validation des professionnels, désactivation, journal, groupes et permissions

Validation **côté serveur** (formulaires et modèles Django : dates passées, doublons, conflits, valeurs impossibles) et **côté client** (champs typés, `required` , `min` / `max` , `datetime-local`), messages d'erreur reliés aux champs. Le périmètre annoncé est couvert, sauf le module laboratoires (équipe de quatre). Parcours inter-modules testé : prescription du médecin → demande du patient en un clic → infirmier → constantes dans le dossier.

2 · Tests & besoins non fonctionnels 4 points

Stratégie de tests

- **Unitaires** : modèles, règles métier, garde-fous IA (IA simulée), formulaires, gabarits
- **Intégration** (`tests_integration.py` , vraie base + client HTTP) : CRUD de chaque entité, inscriptions, parcours complets, statuts HTTP
- **Permissions testées** : matrice 5 rôles × 21 adresses (anonyme → connexion, autre rôle → 403), données d'un autre compte → 404
- **Couverture** : `coverage run manage.py test` → 89 %, rapport HTML publié par la CI

Sécurité (OWASP)

- CSRF sur tout formulaire (page claire si le jeton expire), ORM uniquement
- Mots de passe hachés PBKDF2 + validateurs ; **blocage 15 min après 5 échecs**
- **DEBUG=False imposé en production**, clé forte obligatoire, HTTPS, HSTS, cookies `Secure` / `HttpOnly` , `X-Frame-Options: DENY`
- `check --deploy` sans avertissement ; secrets en variables d'environnement

Accès et robustesse

- `@login_required` + décorateurs de rôle ; **LoginRequiredMixin + PermissionRequiredMixin** pour la gestion des comptes
- Un **groupe Django par rôle** avec ses permissions (mis à jour à chaque `migrate`)
- Pages **400, 403, 404, 500** personnalisées ; IA, e-mail, SMS dans des `try/except` qui journalisent
- Journal des connexions (traçabilité)

3 · Accessibilité web 2 points

Audit Lighthouse 12.8 (rapports dans `docs/lighthouse/`)

Pages auditées (mode production)	Accessibilité	Bonnes pratiques
Accueil, connexion, inscriptions patient et infirmier	100	100
Patient : tableau de bord, demande de soin, analyse du dossier	100	100
Infirmier : tableau de bord, planning · Médecin : file	100	100
Pharmacie : stock · Admin : tableau de bord, comptes	100	100

Ce qui est fait sur toutes les pages

- `header`, `nav` nommée, `main`, `footer` ; un seul `h1`, titres dans l'ordre
- Lien « Aller au contenu », **focus visible**, tout se fait au clavier (Tab, Entrée)
- Icônes décoratives `aria-hidden`, boutons d'icône nommés, `aria-live` sur les résultats asynchrones
- Contrastes ≥ 6,3:1 (AA = 4,5:1), calculés depuis les CSS par `audit_accessibilite`

4 · Intelligence artificielle 4 points

Cas d'usage (besoin réel)	Technique	Métriques mesurées
Triage du texte libre du patient	scikit-learn (TF-IDF + régression logistique), modèle versionné	Précision 92,7 % (1 500 cas), 89,7 % (39 écrits à la main) ; cas graves repérés 96,9 % ; matrice de confusion
Orientation vers une spécialité	scikit-learn	F1 moyen 0,93 sur 30 phrases écrites à la main
Alertes des comptes rendus de soin	Règles + LLM (Groq), JSON validé, citations vérifiées	60 notes annotées : exactitude 65 % → **95 %**, F1 macro 0,64 → **0,95**, rappel « à signaler » 0,50 → **0,85**, 0 fausse alerte
Dossier, pharmacie	LLM : lecture du dossier, substitution, extraction des médicaments	Qualitatif : sortie refusée si hors catalogue, inventée ou non citée ; durée affichée au médecin

Intégration propre

- Client unique (`patients/ia_client.py`) : clé en variable d'environnement, délai max 6 s, cache, quota par compte
- Garde-fous : anonymisation, liste blanche, citation mot pour mot, consentement du patient
- **L'IA ne décide jamais seule** : règles et médecin gardent la main

Performance et asynchronisme

- Temps mesurés : médiane 423 ms, p95 1,9 s par note (Groq)
- **Celery + Redis** : analyse du dossier (2 appels, jusqu'à 12 s) en arrière-plan, **indicateur de chargement** puis résultat ; e-mails en tâche de fond ; relecture IA reprogrammée si l'IA est en panne
- Sans IA ou en panne : message explicite, les règles continuent

5 · UX / UI 1 point

Même charte sur toutes les pages (thème sombre contrasté, composants communs), **responsive** : menu latéral sur ordinateur, barre d'onglets en bas sur mobile ; **menus identiques** pour un même rôle dans tous les modules ; parcours guidés (étapes, confirmations avant suppression, messages de succès et d'erreur).

Démonstration devant le jury

```
docker compose up --build                          # site + worker Celery + Redis + MongoDB → http://localhost:8000
python manage.py test                              # toute la suite (MongoDB local)
coverage run manage.py test && coverage report     # couverture
python manage.py evaluer_alertes                   # métriques de l'IA des soins à domicile
DJANGO_ENV=production ... python manage.py check --deploy   # réglages de production
```